LINUX / SYSTEMS

Permissions & Ownership Visual Guide

rwx bits, octal math, special bits,
ACLs, and umask — who can do what.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/6

Reading ls -l: The 9 Permission Bits

Three triplets: owner, group, others.

```
$ ls -l deploy.sh
-rwxr-xr-- 1 enmanuel devs 2048 Oct 9 deploy.sh
```

rwx	**r-x**	**r--**
owner	group	other

r = read (4) w = write (2) x = execute (1) — '-' means denied

STEP 2/6

Octal Notation: Bit Math

Each triplet is one octal digit: add r(4) + w(2) + x(1).

Bits	Octal	Meaning
rwx	7	full control
rw-	6	read + write
r-x	5	read + execute
r--	4	read only
---	0	nothing

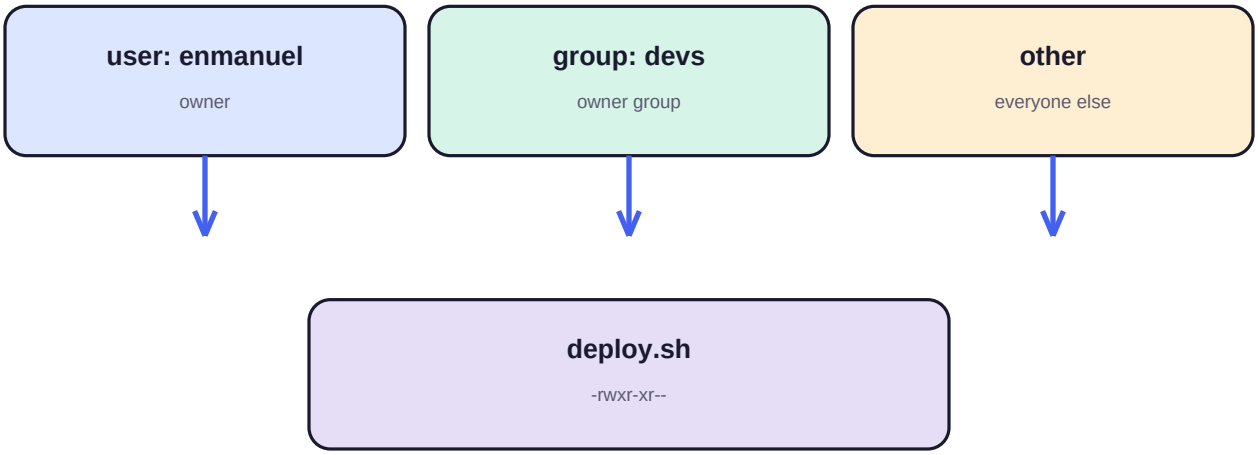
```
$ chmod 755 script.sh    # rwxr-xr-x : runnable by all, writable by owner
$ chmod 644 notes.txt    # rw-r--r-- : classic data file
$ chmod 600 id_rsa       # rw------- : private keys stay private
```

Mnemonic: 7 = do anything, 5 = look but don't touch, 6 = edit but don't run, 0 = locked out.

STEP 3/6

Ownership: Users and Groups

Every file has exactly one owner user and one owner group.




```
$ chown enmanuel:devs deploy.sh     # change owner and group
$ chgrp ops report.csv              # group only
$ chown -R www-data:www-data /srv/www   # recursive
```

Only root (or CAP_CHOWN) can give away ownership. Group membership is the collaboration mechanism — add users to the group, don't chmod 777.

STEP 4/6

Special Bits: setuid, setgid, Sticky

A fourth octal digit for privilege tricks.

Bit	Octal	Symbol	Effect
setuid	4	s in owner-x	run as the FILE OWNER (passwd)
setgid	2	s in group-x	run as file group; dirs: inherit group
sticky	1	t in other-x	/tmp: delete only your own files

```
$ ls -l /usr/bin/passwd
-rwsr-xr-x 1 root root 68208 ... /usr/bin/passwd
$ chmod 4755 prog     # setuid
$ chmod 2750 shared/  # setgid dir: new files inherit group
$ chmod 1777 /tmp     # sticky
```

setuid root binaries are the classic privilege-escalation target — audit them: find / -perm -4000 2>/dev/null.

STEP 5/6

ACLs: Finer Than Owner/Group/Other

Named users and groups get their own entries.

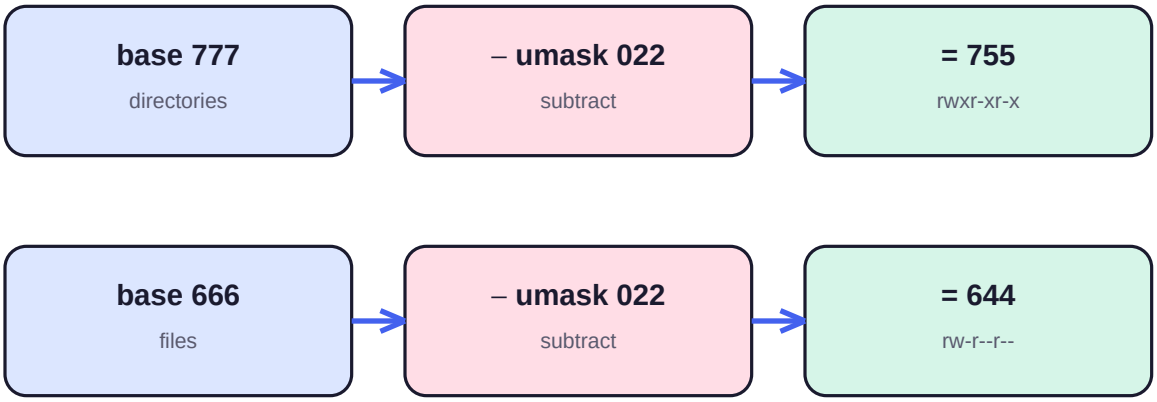
```
$ setfacl -m u:lahiri:rw project.db
$ setfacl -m g:auditors:r report.pdf
$ getfacl project.db
# file: project.db
# owner: enmanuel   # group: devs
user::rw-   user:lahiri:rw-   group::r--   mask::rw-   other::---
```

- Use ACLs when one file needs exceptions beyond a single group — don't spawn groups for one-offs.
- The mask caps every named entry: chmod g-x can silently tighten ACLs. Check getfacl after chmod.
- Default ACLs on a directory auto-apply to new children: setfacl -d -m u:lahiri:rw dir/.

STEP 6/6

umask: Default Permissions for New Files

umask SUBTRACTS from the max: 777 for dirs, 666 for files.




```
$ umask        # 0022
$ umask 027    # stricter: new files 640, new dirs 750
```

Files never get +x from umask math (base is 666) — executability must be granted explicitly with chmod +x.

STEP ✓/✓

Key Takeaways

Permissions & Ownership

- Nine bits in three triplets (owner/group/other); octal adds r=4, w=2, x=1 per triplet.
- Every file has one user owner and one group owner — groups are the collaboration tool.
- Special bits: setuid (run as owner), setgid (run as group / inherit group), sticky (own-files-only delete).
- ACLs grant per-user/per-group exceptions; watch the mask after chmod.
- umask subtracts from 777/666 defaults — new files are never born executable.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev